LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 3

ABSTRACT DATA TYPE (ADT)



Disusun oleh:

Salsadilla Hanny Azizah

109082500014

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 3 : (salsadha/PRAKTIKUMSDT)

1. **Soal 1**

   Source Code

   laprak3_1.main.cpp

```
#include <iostream>
#include <string>
#include "laprak3_1.h"

using namespace std;

int main() {
    Mahasiswa mhs[10];
    int n;

    cout << "Masukkan jumlah data mahasiswa (maks 10): ";
    cin >> n;

    if (n > 10) n = 10;

    for (int i = 0; i < n; i++) {
        cout << "\nData Mahasiswa ke-" << i+1 << endl;
        cout << "Nama  : ";
        cin >> mhs[i].nama;
        cout << "NIM   : ";
        cin >> mhs[i].nim;
        cout << "UTS   : ";
        cin >> mhs[i].uts;
        cout << "UAS   : ";
        cin >> mhs[i].uas;
        cout << "Tugas : ";
        cin >> mhs[i].tugas;

        mhs[i].nilai_akhir = hitungNilai(mhs[i].uts,
mhs[i].uas, mhs[i].tugas);
    }

    cout << "\n=== REKAP NILAI MAHASISWA ===" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Nama: " << mhs[i].nama << " | NIM: " <<
mhs[i].nim
             << " | Nilai Akhir: " << mhs[i].nilai_akhir <<
endl;
    }
```

```
    return 0;
}
```

laprak3_1_mahasiswa.cpp

```
#include "laprak3_1.h"

float hitungNilai(float uts, float uas, float tugas) {
    return (0.3 * uts) + (0.4 * uas) + (0.3 * tugas);
}
```

laprak3_1.h

```
#ifndef MAHASISWA_H_INCLUDED
#define MAHASISWA_H_INCLUDED
#include <string>

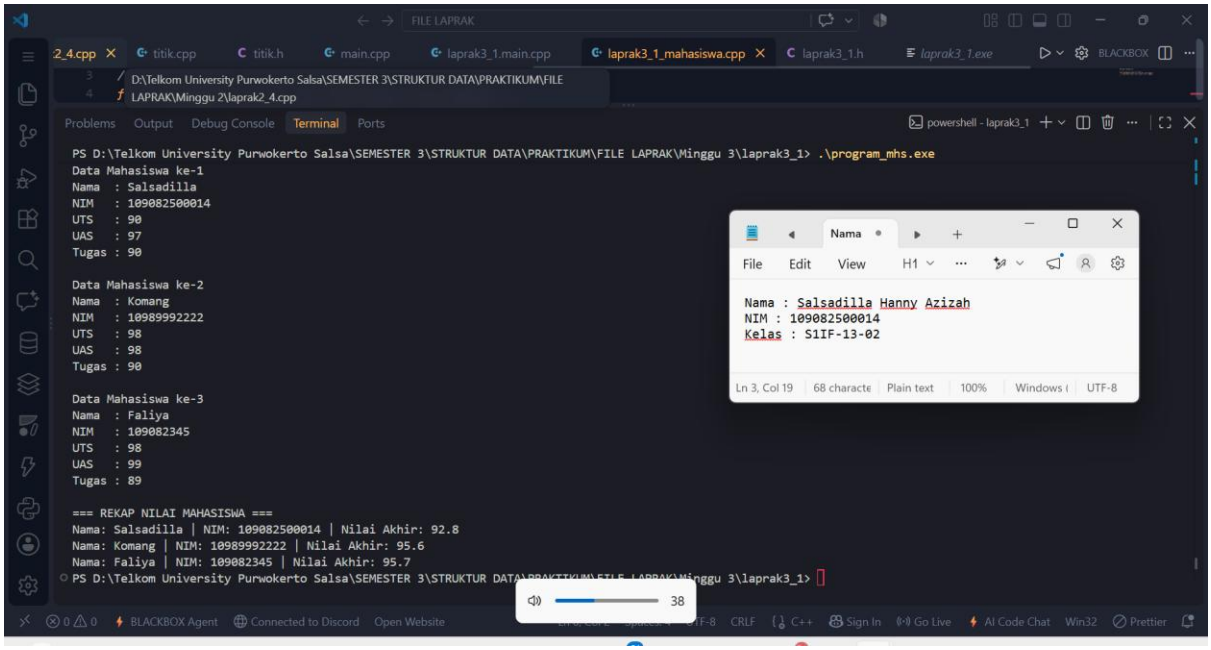
using namespace std;

struct Mahasiswa {
    string nama;
    string nim;
    float uts, uas, tugas, nilai_akhir;
};

float hitungNilai(float uts, float uas, float tugas);

#endif
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mencatat dan menghitung nilai akhir mahasiswa dengan menerapkan konsep *Abstract Data Type* (ADT). Dalam paradigma ini, program dirancang dengan mendefinisikan sebuah tipe data bentukan (TYPE) beserta sekumpulan operasi dasarnya (PRIMITIF) ke dalam modul yang terpisah. Agar mematuhi kaidah ADT, susunan kode sengaja dipecah menjadi tiga file: file *header* .h untuk spesifikasi/deklarasi, file .cpp untuk realisasi rumusnya, dan file main.cpp sebagai program utama yang menjalankannya. Program menggunakan tipe data *array of struct* berkapasitas maksimal 10 elemen untuk menampung rincian data masing-masing mahasiswa. Saat dijalankan, program akan meminta pengguna menginputkan nilai, melempar data tersebut ke modul fungsi perhitungan, lalu mencetak seluruh rekap nilai akhirnya secara berurutan.

Penjelasan Kode Program

- #ifndef, #define, dan #endif Susunan ini disebut sebagai *header guards*. Kegunaannya sangat penting untuk mencegah isi file *header* (laprak3_1.h) mengalami definisi ganda (*error*) apabila file tersebut dipanggil atau disertakan (*include*) lebih dari sekali di dalam program utama.
- struct Mahasiswa { ... }; Merupakan definisi spesifikasi tipe data (TYPE) dari ADT. Struktur (*struct*) ini bertugas mengelompokkan variabel-variabel dasar seperti nama, nim, uts, uas, tugas, dan nilai_akhir agar menyatu ke dalam satu entitas yang sama bernama Mahasiswa.
- float hitungNilai(...) Ini adalah primitif berupa fungsi yang deklarasinya didaftarkan di file *header*, tetapi bodi kode aslinya (realisasi) dipisah ke file laprak3_1_mahasiswa.cpp. Fungsi ini bertugas menerima parameter bobot nilai, menghitungnya sesuai rumus, lalu memiliki nilai balik (*return*) bertipe *float*.

- #include "laprak3_1.h" Baris perintah ini wajib diselipkan pada bagian atas file realisasi dan file main.cpp agar kedua file tersebut saling terhubung dan mengenali *struct* serta fungsi primitif yang sudah kita buat di dalam file *header*.
- Mahasiswa mhs[10]; Mendeklarasikan sebuah array bernama mhs menggunakan tipe data Mahasiswa dengan batasan kapasitas maksimum 10 elemen. Artinya, memori menyiapkan ruang untuk menampung hingga 10 data mahasiswa yang berbeda-beda.
- for(int i = 0; i < n; i++) Struktur perulangan (*looping*) ini dipakai dua kali di program utama. *Looping* yang pertama dipakai untuk mengarahkan pengguna menginputkan data ke masing-masing elemen array secara berurutan. *Looping* yang kedua khusus bertugas menyisir ulang isi array untuk mencetak kembali seluruh data yang sudah direkap ke layar terminal.
- mhs[i].nilai_akhir = hitungNilai(...)**;** Program utama memanggil primitif fungsi perhitungan dengan mengirimkan data nilai UTS, UAS, dan tugas khusus milik mahasiswa ke-i. Hasil kalkulasinya akan seketika itu juga disimpan ke dalam *field* nilai_akhir miliknya.

2. Soal 2

Source Code

pelajaran.h

```
#ifndef PELAJARAN_H_INCLUDED
#define PELAJARAN_H_INCLUDED
#include <string>

using namespace std;

struct pelajaran {
    string namamapel;
    string kodemapel;
};

pelajaran create_pelajaran(string namapel, string kodepel);
void tampil_pelajaran(pelajaran pel);

#endif
```

pelajaran.cpp

```
#include <iostream>
#include <string>
#include "pelajaran.h"

using namespace std;
```

```
pelajaran create_pelajaran(string namapel, string kodepel) {
    pelajaran pel;
    pel.namamapel = namapel;
    pel.kodemapel = kodepel;
    return pel;
}

void tampil_pelajaran(pelajaran pel) {
    cout << "nama pelajaran: " << pel.namamapel << endl;
    cout << "nilai: " << pel.kodemapel << endl;
}
```

main.cpp

```
#include <iostream>
#include <string>
#include "pelajaran.h"

using namespace std;

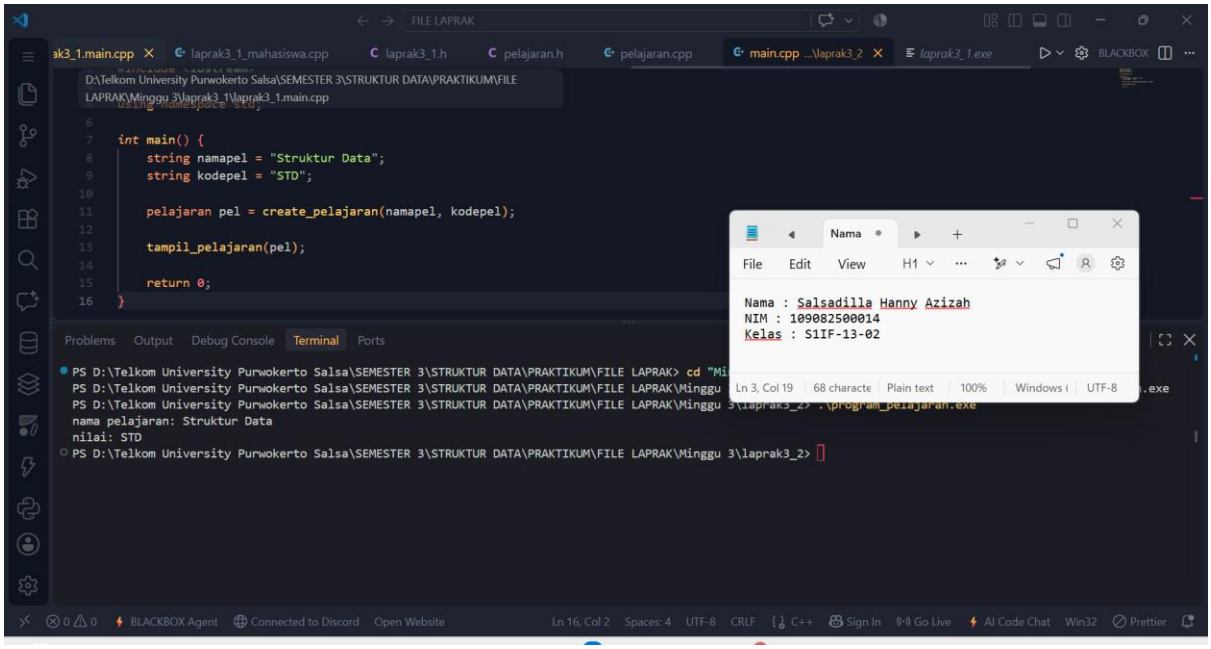
int main() {
    string namapel = "Struktur Data";
    string kodepel = "STD";

    pelajaran pel = create_pelajaran(namapel, kodepel);

    tampil_pelajaran(pel);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mengimplementasikan konsep *Abstract Data Type* (ADT) statik dengan memecah susunan kode ke dalam tiga file terpisah, yaitu modul definisi (.h), modul realisasi (.cpp), dan program utama (main.cpp). Pada program ini, dibuat sebuah tipe data bentukan bernama pelajaran yang khusus menyimpan data nama dan kode mata pelajaran. Alurnya cukup sederhana: program utama memanggil fungsi pembentuk (kreator) untuk mengisi data pelajaran yang sudah disiapkan, lalu memanggil prosedur untuk menampilkannya ke layar terminal.

Penjelasan Kode Program

- #ifndef, #define, dan #endif Susunan ini diletakkan di file pelajaran.h sebagai *header guards* untuk mencegah *error* akibat isi file terdefinisi ganda jika tidak sengaja dipanggil berulang kali di program utama.
- struct pelajaran { ... }; Mendefinisikan spesifikasi tipe data (TYPE) dari ADT. *Struct* ini bertugas mengelompokkan variabel namamapel dan kodemapel yang bertipe *string* ke dalam satu kesatuan entitas.
- pelajaran create_pelajaran(...) Merupakan fungsi pembuat nilai (kreator) yang logikanya dijabarkan di file pelajaran.cpp. Fungsi ini menerima parameter nama dan kode pelajaran, menyimpannya ke dalam *struct* lokal, lalu mengembalikan data hasil rakitannya ke pemanggil.
- void tampil_pelajaran(...) Sebuah prosedur yang bertugas khusus sebagai perantara baca/tulis ke layar (*interface*). Prosedur ini murni hanya mencetak teks nama dan kode mapel ke layar terminal tanpa memiliki nilai balik (*return value*).
- #include "pelajaran.h" Baris pemanggilan *header* ini wajib diletakkan pada file realisasi pelajaran.cpp dan file main.cpp agar keduanya terhubung dan mengenali rancangan tipe data beserta fungsi ADT yang sudah kita daftarkan sebelumnya.

- pelajaran pel = create_pelajaran(...); Berada di dalam fungsi utama (main.cpp), baris ini mendeklarasikan variabel pel sekaligus memanggil fungsi kreator dengan mengirimkan teks "Struktur Data" dan "STD" untuk langsung diisi ke dalam *struct* tersebut.

3. Soal 3

Source Code

```
#include <iostream>
using namespace std;

// Prosedur buat nampilin matriks/array 2D ke layar
void tampilArray(int arr[3][3]) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
}

// Prosedur buat nuker isi array di koordinat tertentu
void tukarIsiArray(int arr[3][3], int b1, int k1, int b2, int
k2) {
    int temp = arr[b1][k1]; // Titipin dulu nilainya ke 'temp'
biar nggak hilang
    arr[b1][k1] = arr[b2][k2];
    arr[b2][k2] = temp;
}

// Prosedur buat nuker isi nilai memori dari 2 pointer
void tukarPointer(int *p1, int *p2) {
    int temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}

int main() {
    int arr1[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int arr2[3][3] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};

    int a = 10, b = 20;
    int *ptr1 = &a;
    int *ptr2 = &b;

    cout << "=== Tampil Array 1 Awal ===" << endl;
    tampilArray(arr1);
```

```
    cout << "\n=== Menukar arr1[0][0] dengan arr1[2][2] ===" <<
endl;
    tukarIsiArray(arr1, 0, 0, 2, 2);
    tampilArray(arr1);

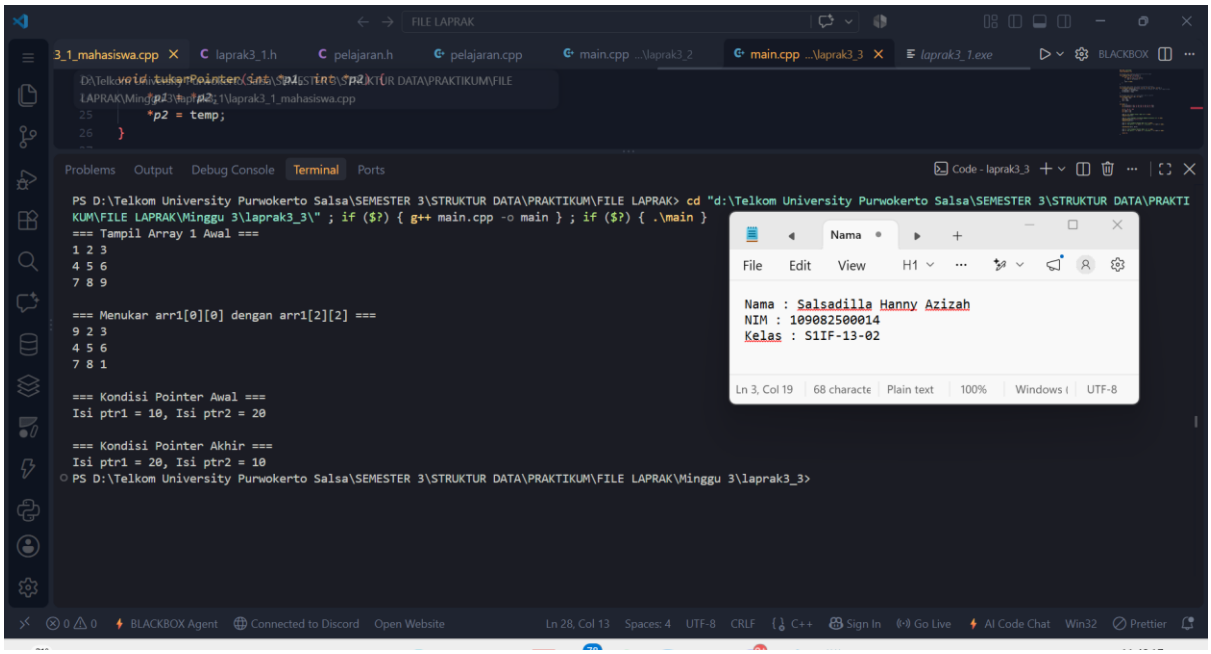
    cout << "\n=== Kondisi Pointer Awal ===" << endl;
    cout << "Isi ptr1 = " << *ptr1 << ", Isi ptr2 = " << *ptr2
<< endl;

    tukarPointer(ptr1, ptr2);

    cout << "\n=== Kondisi Pointer Akhir ===" << endl;
    cout << "Isi ptr1 = " << *ptr1 << ", Isi ptr2 = " << *ptr2
<< endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Program ini dibuat untuk mempraktikkan cara manipulasi data di dalam array dua dimensi sekaligus mencoba operasi penukaran nilai menggunakan pointer. Alurnya berpusat di fungsi utama yang mendeklarasikan dua buah array ukuran 3x3 beserta dua variabel pointer. Setelah itu, program bakal manggil tiga prosedur berbeda yang punya tugasnya masing-masing: prosedur pertama khusus buat mencetak isi matriks ke layar terminal,

prosedur kedua buat nuker posisi dua angka di dalam array, dan prosedur ketiga dipakai buat nuker isi nilai memori yang lagi ditunjuk sama pointer.

Penjelasan Kode Program

- void tampilArray(int arr[3][3]) Prosedur ini dipakai buat nyetak array 2D ke layar terminal. Di dalamnya pakai perulangan bersarang (*nested loop*) for dengan variabel i buat baris dan j buat kolom biar angkanya bisa diakses dan tercetak rapi berbentuk matriks.
- void tukarIsiArray(...) Bagian ini adalah prosedur (nggak punya *return value*) yang dirancang khusus buat nge-*swap* (nuker) dua data di koordinat array yang beda. Logikanya pakai variabel bantuan temp (sementara) buat nitip nilai angka pertamanya biar nggak hilang pas ditimpa pakai angka kedua.
- void tukarPointer(int *p1, int *p2) Prosedur ini pakai parameter pointer (ada tanda bintang *). Pas proses tuker-tukeran di dalamnya, kita wajib nambahin simbol bintang lagi di depan nama pointernya (misal *p1) biar yang ketukar itu murni nilai asli yang ada di memorinya, bukan alamatnya.
- int arr1[3][3] = { ... }; Baris ini buat bikin array dua dimensi ukurannya 3x3 dan langsung diisi (diinisialisasi) dengan deretan angka urut dari 1 sampai 9.
- int *ptr1 = &a; Ini dipakai buat bikin variabel pointer ptr1 yang ditugasin nyimpen alamat memori dari variabel a. Tanda *ampersand* (&) di depan nama variabel a itu fungsinya buat ngambil alamat asli variabel tersebut.

LATIHAN MODUL 3 (UNGUIDED)

4. Soal UNGUIDED

Source Code

main.cpp

```
#include "titik.h"
#include <iostream>

using namespace std ;

int main (){
    Titik tA, tB ;

    cout << "--- Input Titik Pertama ---" << endl;
    inputTitik(tA);

    cout << "--- Input Titik Kedua ---" << endl;
    inputTitik(tB);

    cout << "\nHasil Rekap Kordinat" << endl;
    tampilTitik(tA);
    tampilTitik(tB);

    float jarak = hitungJarak(tA, tB);

    cout << " Jarak antara kedua titik: " << jarak << endl;

    return 0;
}
```

titik.cpp

```
#include "titik.h"
#include <cmath>
#include <iostream>

using namespace std;

void inputTitik(Titik &t){
    cout << "Masukan Kordinat X: ";
    cin >> t.x;
    cout << "Masukan Kordinat Y: ";
    cin >> t.y;
}
```

```
void tampilTitik(Titik t){
    cout << "Posisi titik : (" << t.x <<", "<< t.y <<") " <<
endl;
}

float hitungJarak(Titik t1, Titik t2){
    return sqrt(pow(t2.x - t1.x, 2) + pow(t2.y - t1.y, 2));
}
```

titik.h

```
#ifndef TITIK_H_INCLUDE
#define TITIK_H_INCLUDE
#include <iostream>
using namespace std;

struct Titik {
    float x;
    float y;
};
void inputTitik(Titik &t);
void tampilTitik(Titik t);
float hitungJarak(Titik t1, Titik t2);

#endif
```

Deskripsi program

Program ini dibuat untuk mengimplementasikan konsep *Abstract Data Type* (ADT) pada entitas koordinat atau titik. Sesuai dengan kaidah pemrograman modular ADT statik, struktur kodenya sengaja dipecah menjadi tiga file terpisah, yaitu modul definisi (titik.h), modul realisasi (titik.cpp), dan program utama (main.cpp). Alur kerja program ini cukup sederhana: program akan meminta pengguna untuk memasukkan koordinat X dan Y untuk dua buah titik yang berbeda. Setelah data dimasukkan, program akan memanggil prosedur untuk menampilkan kembali posisi kedua titik tersebut, lalu memanggil fungsi matematika untuk menghitung dan mencetak jarak pasti antara keduanya.

Penjelasan Kode Program

- #ifndef, #define, dan #endif Susunan ini diletakkan di dalam file titik.h sebagai *header guards*. Kegunaannya sangat krusial untuk mencegah *error* akibat definisi ganda apabila file *header* ini dipanggil lebih dari sekali di program utama.
- struct Titik { ... }; Merupakan pembuatan spesifikasi tipe data (TYPE) dari ADT. *Struct* ini bertugas membungkus atau mengelompokkan variabel x dan y yang bertipe *float* (desimal) ke dalam satu kesatuan bernama Titik.

- void inputTitik(Titik &t) Ini adalah prosedur yang bertugas meminta input nilai dari pengguna. Pada parameternya sengaja ditambahkan tanda *ampersand* (&) yang menandakan metode *Call by Reference*, sehingga data yang diinputkan di dalam prosedur ini akan benar-benar tersimpan dan mengubah memori variabel aslinya.
- void tampilTitik(Titik t) Prosedur ini khusus dipakai untuk mencetak koordinat ke layar terminal. Karena fungsinya hanya sekadar membaca tanpa perlu mengubah data aslinya, maka parameternya cukup menggunakan *Call by Value* biasa tanpa tambahan tanda &.
- float hitungJarak(...) Sebuah fungsi primitif yang dirancang untuk memiliki nilai kembalian (*return value*) bertipe *float*. Logikanya menggunakan rumus jarak Euclidean, di mana perhitungannya dibantu oleh *library* tambahan <cmath> agar kita bisa menggunakan fungsi pow untuk pangkat dan sqrt untuk menarik akar.
- #include "titik.h" Baris pemanggilan ini wajib diletakkan di bagian atas file titik.cpp dan main.cpp dengan menggunakan tanda kutip ganda (" "). Fungsinya agar kedua file tersebut saling terhubung dan mengenali *struct* beserta purwarupa fungsi yang telah kita daftarkan di file lokal *header* tersebut.
- Titik tA, tB; Dideklarasikan di dalam fungsi utama, baris ini menyiapkan dua buah variabel bernama tA dan tB menggunakan tipe data bentukan Titik yang sudah kita buat.